

운영체제 5

남궁명수

[1-1] 병행성: 세마포어	2
[1] 세마포어 정의	2
[2] 이진 세마포어(락)	4
[3] 조건 변수로서의 세마포어	6
[4] 생산자/소비자(유한버퍼) 문제	9
[5] Reader-Writer 락	16
[6] 식사하는 철학자	19
[7] 세마포어 구현	22
[1-2] 병행성: 병행성 관련 오류	24
[1] 오류의 종류	24
[2] 비 교착 상태 오류	24
[4] 요약	28
[1-3] 병행성: 이벤트 기반 병행성(Event-Based Concurrency)	29
[1] 핵심 구조: 이벤트 루프	29
[2] 핵심 API: select() / poll()	29
[3] select()의 사용	31
[4] 왜 간단한가? 락이 필요 없다	32
[5] 문제: 블로킹 시스템 콜	32
[6] 해법: 비동기 I/O	33
[7] 또 다른 문제점: 상태 관리	34
[8] 이벤트 사용의 어려움	35
[9] 요약	36

[1-1] 병행성: 세마포어

핵심 질문: 세마포어를 어떻게 사용하는가

- 락과 컨디션 변수 대신 세마포어를 사용하는 방법은 무엇인가?
- 세마포어의 정의는 무엇인가?
- 이진 세마포어는 무엇인가?
- 락과 컨디션 변수를 사용하여 세마포어를 만드는 것이 가능한가?
- 그 반대로 세마포어를 사용하여 락과 조건 변수를 만드는 것이 가능한가?

다양한 범주의 병행성 문제 해결을 위해서는 락과 조건 변수가 모두 필요합니다. 정확한 역사를 파악하기는 어렵지만, 이 사실을 최초로 인지한 사람 중에 Edsger Dijkstra 가 있다.

그는 “최단 경로” 알고리즘이라고 하는 그래프 이론으로 유명할 뿐만 아니라, “Goto”문의 유해성이라는 구조화 프로그래밍에 관한 격렬한 논쟁을 이끌어낸 것으로도 유명하다.

이번 장에서 다루게 될 세마포어(semaphore)라는 동기화 기법도 그가 개발한 것이다.

Dijkstra 와 그의 동료들은 모든 다양한 동기화 관련 문제를 한 번에 해결할 수 있는 기법을 개발하고자 했다.

그리고 세마포어가 탄생했다.

실로 엄청난 사건이 아닐 수 없다. 이제 보게 되겠지만, 세마포어는 락과 컨디션 변수로 모두 사용할 수 있다.

[1] 세마포어 정의

세마포어는 정수 값을 갖는 객체로서 두 개의 루틴으로 조작할 수 있다.

POSIX 표준에서 이 두개의 루틴은 `sem_wait()`와 `sem_post()`이다.

세마포어는 이 초기값에 의해 동작이 결정되기 때문에, 사용하기 전에 “제일 먼저” 값을 초기화 해야 한다.

역사적으로 Dijkstra 는 `sem_wait()`를 P()로, `sem_post()`를 V()로 칭했다.

P()는 네덜란드어로 “시도하다”라는 뜻을 가진 “prolaag”와 “probeer”의 줄임말이다.

V()는 네덜란드어로 “증가하다(increase)”라는 뜻을 가진 “verhoog”에서 유래한다.

때로는 down 과 up 으로 표현하기도 한다.

```
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 1);
```

위 코드에서 세마포어 `s`를 선언 후, 3 번째 인자로 1 을 전달하여 세마포어의 값을 1 로 초기화한다.

`sem_init()`의 두 번째 인자는 모든 예제에서 0 이다.

이 값은 같은 프로세스 내의 스레드 간에 세마포어를 공유한다는 것을 의미한다.

두 번째 인자에서 다른 값을 사용 하는 예(다른 프로세스 간에 동기화를 제공하는 방법 등)는 세마포어 용법에 대한 메뉴얼을 읽어보기를 권한다.

초기화된 후에는 `sem_wait()` 또는 `sem_post()`라는 함수를 호출하여 세마포어를 다룰 수 있다.

```
int sem_wait(sem_t *s) {
    세마포어 s 의 값을 1 감소시킨다;
    세마포어 s 의 값이 음수이면 대기한다;
}

int sem_post(sem_t *s) {
    세마포어 s 의 값을 1 증가시킨다;
    하나 이상의 스레드가 대기 중이면, 그중 하나를 깨운다;
}
```

이 루틴들은 다수 스레드들에 의해 동시에 호출되는 것을 가정한다.

임계 영역은 적절히 보호되어야 한다.

임계 영역 보호에 대한 고민은 잠시 후로 미루자. 지금은 이 두 함수의 사용 방법에 집중하자.

우선 몇 가지 핵심적인 성질을 논의해 보자.

첫 번째로, `sem_wait()` 함수는 즉시 리턴하거나(세마포어의 값이 1 이상이면), 아니면 해당 세마 포어 값이 1 이상이 될 때까지 호출자를 대기시킨다.

다수의 스레드들이 `sem_wait()`를 호출할 수 있기 때문에, 대기 큐에는 다수의 스레드가 존재할 수 있다.

대기하는 방법에는 회전(spin)과 재우기(sleep)의 두 가지가 있다는 것을 상기하자.

두 번째, sem_wait() 함수와 달리 sem_post() 함수는 대기하지 않는다.
세마포어 값을 증가시키고 대기 중인 스레드 중 하나를 깨운다.

세 번째로, 세마포어가 음수라면, 그 값은 현재 대기 중인 스레드의 개수와 같다.
일반적으로 세마포어 사용자는 이 값을 알 필요가 없다.
하지만 이러한 성질을 알고 있는 것이 세마포어 작동을 이해하는데 도움이 된다.

이 두개의 함수는 원자적으로 실행된다고 가정한다.
세마포어 루틴 내에서 레이스 조건이 발생할 수 있다는 사실은(아직은) 걱정하지
말자.
그것을 해결하기 위해 곧 락과 조건 변수를 사용하게 될 것이다.

[2] 이진 세마포어(락)

이제 세마포어를 사용할 준비가 되었다. 우리가 처음으로 세마포어를 적용할 곳은
이미 친숙한 “락”이다.

```
sem_t m;  
sem_init(&m, 0, X); // 초기값 X  
sem_wait(&m);  
// 임계영역  
sem_post(&m);
```

sem_wait()와 sem_post()의 정의를 되새겨보면 초기값은 1 이 되어야 한다는 것을 알
수 있다.

이 부분을 분명히 하기 위해 스레드가 두 개인 경우를 생각해 보자.

첫 스레드(스레드 0)가 sem_wait()를 호출한다. 먼저 세마포어 값을 1 감소시켜
0 으로 만든다.

스레드는 세마포어 값이 음수인 경우에만 대기한다. 세마포어의 값이 0 이므로
리턴하고 진행할 수 있다.

스레드 0 은 이제 임계 영역에 진입할 수 있다.

스레드 0 이 임계 영역 내에 있을 때 다른 스레드가 락을 획득하려고 하지 않는다면,
이 스레드가 sem_post()를 불렀을 때 세마포어 값이 다시 1 로 설정된다.(대기 중인
스레드가 없기 때문에 아무도 깨우지 않는다)

단계	세마포어 값	스레드 0 (T0)	스레드 1 (T1)
초기 상태	1	-	-
1	1 → 0	sem_wait() 호출	-
2	0	sem_wait() 리턴 (진입)	-
3	0	임계 영역 실행 중	-
4	0 → 1	sem_post() 호출	-
5	1	sem_post() 리턴 (종료)	-

좀 더 흥미로운 상황은 스레드 0 이 “락을 보유하고” 있을 때(sem_wait()를 호출하여 임계영역에 진입했지만 아직 sem_post()를 호출하지 않은 경우)
다른 스레드(스레드 1)가 sem_wait()를 호출하여 임계 영역 진입을 시도하는
경우이다.

이 경우 스레드 1 이 세마포어 값을 -1 로 감소시키고 대기에 들어간다.(프로세서를 내어주고 스스로 잠자기에 들어간다.)

스레드 0 이 다시 실행되면 sem_post()를 호출하고, 세마포어 값을 0 으로 증가시키고, 잠자던 스레드(스레드 1)를 깨운다.

그러면 스레드 1 이 락을 획득할 수 있게 된다. 스레드 1 의 작업이 끝나면, 세마포어의 값을 다시 증가시켜 1 이 되도록 한다.

혼자 좀 더 공부하고자 한다면, 하나의 락을 여러 개의 스레드가 기다리는 경우를
생각해보자.

그 상황에서는 세마포어의 값이 어떻게 변화할까?

세마포어를 락으로 쓸 수 있다는 것을 알았다. 락은 두 개의 상태(사용 가능, 사용
중)만 존재하므로 이진 세마포어라고도 불린다.

[3] 컨디션 변수로서의 세마포어

어떤 조건이 참이 되기를 기다리기 위해 현재 스레드를 멈출 때에도 세마포어는 유용하게 사용될 수 있다.

예를 들어, 리스트에서 객체를 삭제하기 위해 리스트에 객체가 추가되기를 대기하는 스레드가 있을 수 있다.

이런 종류의 전형적인 패턴이 하나의 스레드가 어떤 사건을 발생을 기다리고, 또 다른 스레드는 해당 사건을 발생시킨 후, 시그널을 보내어 기다리는 스레드를 깨우는 것이다.

대기 중인 스레드(또는 스레드들)가 프로그램에서의 어떤 조건이 만족되기를 대기하기 때문에, 세마포어를 컨디션 변수처럼 사용할 수 있다.

다음과 같은 간단한 예제를 보자.

부모 스레드가 자식 스레드를 생성한 후, 자식 스레드의 종료를 대기하고자 한다.

이 프로그램이 실행하여 다음과 같은 결과를 얻고자 한다.

```
parent: begin
child
parent: end
```

```
sem_t s;
void *child(void *arg){
    printf("child\n");
    sem_post(&s) // 자신 완료 신호
    return NULL;
}

int main(int argc, char *argv[]){
    sem_init(&s, 0, X); // 초기값 X
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(c, NULL, child, NULL);
    sem_wait(&s); // 자식 대기
    printf("parent: end\n");
    return 0;
}
```

세마포어를 이용하여 어떻게 이 효과를 만들 수 있을까?

답은 의외로 간단하다.

코드에서 볼 수 있듯이, 부모 프로세스는 자식 프로세스 생성 후 `sem_wait()`를 호출하여 자식의 종료를 대기한다.

자식은 `sem_post()`를 호출하여 종료되었음을 알린다.

중요한 질문이 있다. 세마포어 값을 무엇으로 초기화할까?

- 정답: 세마포어의 초기값은 0 이다. 두가지 상황이 발생할 수 있다.

첫 번째 경우:

자식 프로세스 생성 후, 아직 자식 프로세스가 실행을 시작하지 않은 경우다.
이 경우 자식이 `sem_post()`를 호출하기 전에 부모가 `sem_wait()`를 호출할 것이다.
부모 프로세스는 자식이 실행될 때까지 대기해야 한다.
이를 위해서 `wait()` 호출 전에 세마포어 값을 감소시키고(-1 로)대기한다.
자식이 실행되었을 때 `sem_post()`를 호출하여 세마포어의 값을 0 으로 증가시킨 후 부모를 깨운다.
그러면 부모는 `sem_wait()`에서 리턴을 하여 프로그램을 종료시킨다.

1. 초기값 = 0
2. 부모가 `sem_wait()` 호출
 - 0 → -1
 - 음수 → 기다림(block)
3. 자식이 나중에 실행됨
4. 자식이 `sem_post()` 호출
 - -1 → 0
 - 부모 깨움
5. 부모 깨어나서 계속 실행

두 번째 경우:

부모 프로세스가 `sem_wait()`를 호출하기 전에 자식 프로세스의 실행이 종료된 경우이다.

이 경우 자식이 먼저 `sem_post()`를 호출하여 세마포어 값을 0에서 1로 증가시킨다. 부모가 실행할 수 있는 상황이 되면 `sem_wait()`를 호출한다.

세마포어 값이 1인것을 발견할 것이다.

부모는 세마포어 값을 0으로 감소시키고 `sem_wait()`에서 대기 없이 리턴한다.

이 방법 역시 의도한 결과를 만들어 낸다.

1. 초기값 = 0
2. 자식이 먼저 실행
3. 자식이 `sem_post()` 호출
 - 0 → 1
4. 나중에 부모가 `sem_wait()` 호출
 - 1 → 0
 - 대기 안하고 바로 통과

[4] 생산자/소비자(유한버퍼) 문제

다음 문제는 생산자/소비자 문제 또는 유한 버퍼 라고 불리는 문제이다.

이 문제는 이전 장에서 상세히 설명하였으므로 구체적인 내용은 이전 장을 참고하길 바란다.

첫 번째 시도

이 문제를 해결하는 첫 번째 시도에는 empty 와 full 이라는 두 개의 세마포어를 사용한다.

스레드는 empty 와 full 을 사용하여 버퍼 공간이 비는지 채워졌는지를 표시한다.

```
int buffer[MAX];
int fill = 0;    // 넣는 위치
int use = 0;    // 꺼내는 위치

void put(int value) {
    buffer[fill] = value;    // f1
    fill = (fill + 1) % MAX; // f2
}

int get() {
    int tmp = buffer[use];    // g1
    use = (use + 1) % MAX;    // g2
    return tmp;
}
```

전체 구조 이해하기

```
int buffer[MAX];
int fill = 0;
int use = 0;
```

- buffer: 데이터를 저장하는 배열
- fill: 다음에 데이터를 넣을 위치
- use: 다음에 데이터를 꺼낼 위치

put()함수(데이터 넣기 = 생산자)

```
buffer[fill] = value; //f1  
fill = (fill + 1) % MAX; // f2
```

동작과정

1. buffer[fill]위치에 값 저장
2. fill 을 다음 칸으로 이동

중요한 부분

```
fill = (fill + 1) % MAX;
```

- 배열 끝까지 가면 다시 처음으로 돌아옴
- 즉 원형 구조

예시

```
MAX = 5  
fill -> 4 -> (4+1)%5=0
```

get()함수(데이터 꺼내기 = 소비자)

```
int tmp = buffer[use]; // g1  
use = (use + 1) % MAX;  
return tmp;
```

1. buffer[use]에서 값 꺼냄
2. use 를 다음 칸으로 이동
3. 값 반환

```

sem_t empty; // 빈 칸 개수
sem_t full; // 채워진 칸 개수

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // P1
        put(i);           // P2
        sem_post(&full);  // P3
    }
}

void *consumer(void *arg) {
    int i, tmp = 0;
    while (tmp != -1) {
        sem_wait(&full); // C1
        tmp = get();     // C2
        sem_post(&empty); // C3
        printf("%d\n", tmp);
    }
}

int main(int argc, char *argv[]) {
    sem_init(&empty, 0, MAX); // MAX 로 초기화
    sem_init(&full, 0, 0);    // 0 으로 초기화
}

```

- 생산자: 데이터 넣기(put)
- 소비자: 데이터 꺼내기(get)
- 문제
 - 버퍼 넘침 방지
 - 빈 버퍼 접근 방지
 - 동시에 접근 충돌 방지

사용되는 세마포어

- empty: 빈 공간 개수, MAX
- full: 채워진 데이터 개수, 0

생산자(Producer)

```
sem_wait(&empty); //빈 공간 확인
put(i);           // 데이터 삽입
sem_post(&full);  // 데이터 생겼다고 알림
```

1. 빈 자리 있어야 넣음
2. 데이터 넣음
3. 소비자에게 “데이터 있음” 알림

소비자(Consumer)

```
sem_wait(&full); // 데이터 있는지 확인
tmp = get();     // 데이터 꺼냄
sem_post(&empty); // 빈자리 생겼다고 알림
```

1. 데이터 있어야 꺼냄
2. 데이터 꺼냄
3. 생산자에게 “자리 남음” 알림

전체 흐름

[Producer]		[Consumer]
empty ↓		full ↓
put()	----->	get()
full ↑		empty ↑

이 예제에서 생산자는 버퍼가 비워져서 데이터를 넣을 수 있기를 기다리고, 마찬가지로 소비자는 데이터를 꺼내기 위해 버퍼가 채워지기를 기다린다.

소비자가 먼저 실행했다고 가정하면 소비자 스레드가 C1 라인에 도달하여 `sem_wait(&full)`을 호출한다.

변수 `full`의 값은 0으로 초기화되기 때문에 해당 명령으로 인해 `full`의 값은 -1로 감소되고, 소비자는 대기한다.

그런 이후에 생산자 스레드가 실행하여 P1 라인에서 `sem_wait(&empty)`를 호출한다. `empty` 변수가 MAX(이 경우에는 1)로 설정되기 때문에 소비자와 다르게 생산자는 다음 문장을 계속 실행한다.

`empty` 변수는 감소하여 0이 되고 생산자가 데이터 값을 버퍼의 첫 번째 공간에 넣는다.(P2 라인).

그런후에 생산자는 P3 라인의 `sem_post(&full)`를 호출하여 세마포어 `full`의 값을 -1에서 0으로 변경하고 소비자 스레드를 깨운다.

이번에는 MAX 값이 1 보다 크고 (MAX = 10), 또한 생산자와 소비자 스레드들이 여러 개 있다고 하자.

경쟁 조건이 발생한다.

어디에서 경쟁이 발생하는지 찾을 수 있겠는가? 힌트는 put()과 get()코드를 유심히 살펴봐야 한다.

두 개의 생산자 Pa 와 Pb 가 있는데, 두 스레드가 put()을 거의 동시에 호출하였다고 해보자.

Pa 가 먼저 실행되어서 버퍼의 첫 공간에 값을 넣기 시작한다.(f1 라인에서 fill = 0 이다.)

Pa 스레드가 fill 카운터 변수를 1 로 변경하기 전에 인터럽트가 걸렸다.

생산자 Pa 가 실행되고 f1 라인에서 마찬가지로 버퍼의 첫 번째 공간에 데이터를 삽입한다.

Pa 가 기록한 이전의 값은 새로운 값으로 대체된다.!

이렇게 하면 절대 안된다. 생산자의 데이터 중 어느것도 사라지면 안 된다.

해답: 상호배제의 추가

위에서 상호배제를 고려하지 않았다. 버퍼를 채우고 버퍼에 대한 인덱스를 증가하는 동작은 임계 영역이기 때문에 신중하게 처리해야 한다.

이진 세마포어와 몇 개의 락을 추가하여 해결해 보자.

```
sem_t empty;
sem_t full;
sem_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex); // p0
        sem_wait(&empty); // p1
        put(i);           // p2
        sem_post(&full);  // p3
        sem_post(&mutex); // p4
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
```

```

        sem_wait(&mutex); // c0
        sem_wait(&full); // c1
        int tmp = get(); // c2
        sem_post(&empty); // c3
        sem_post(&mutex); // c4
        printf("%d\\n", tmp);
    }
}

int main(int argc, char *argv[]) {
    sem_init(&empty, 0, MAX);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
}

```

이제 put()/get() 코드에 락을 추가했다.

괜찮은 발상인 것 같은데, 아직도 동작하지 않는다. 왜냐하면 교착 상태이다.

교착 상태의 방지

생산자와 소비자 스레드가 각 하나씩 있다고 하자.

소비자가 먼저 실행이 된다.

mutex(c0 라인)를 획득하고 full 변수에 대하여 sem_wait()(c1 라인)를 호출한다.

아직 데이터가 없기 때문에 소비자는 대기해야 하고 CPU를 양보해야 한다.

여기서 중요한 것은 소비자가 아직도 락을 획득하고 있다는 것이다.

생산자가 실행된다.

실행이 가능하면 데이터를 생성하고 소비자 스레드를 깨울 것이다.

불행하게도 이 스레드는 먼저 mutex 세마포어에 대해서 sem_wait()를 실행한다.(p0 라인).

이미 락은 소비자가 획득한 상태이기 때문에 생산자 역시 대기에 들어간다.

순환 고리가 생겼다.

소비자는 mutex를 갖고 있으면서 full 시그널이 발생하기를 대기하고 있다.

full 시그널을 발생시켜야 하는 생산자는 mutex에서 대기 중이다.

생산자와 소비자가 서로를 기다린다. 전형적인 교착상태이다.

최종, 제대로 된 해법

이 문제를 해결하기 위해서는 락의 범위(scope)를 줄여야 한다.

```
sem_t empty;
sem_t full;
sem_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // p1
        sem_wait(&mutex); // p1.5 MUTEX
        put(i);           // p2
        sem_post(&mutex); // p2.5
        sem_post(&full);  // p3
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&full); // c1
        sem_wait(&mutex); // c1.5 MUTEX
        int tmp = get(); // c2
        sem_post(&mutex); // c2.5
        sem_post(&empty); // c3
        printf("%d\n", tmp);
    }
}

int main(int argc, char *argv[]) {
    sem_init(&empty, 0, MAX);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
}
```

보는 바와 같이, mutex 를 획득하고, 해제하는 코드를 임계 영역 바로 이전과 이후로 이동하였다.

full 이나 empty 와 관련된 코드는 mutex 밖으로 배치하였다.

그 결과 멀티 스레드 프로그램에서 사용 가능한 유한 버퍼를 만들었다.

앞으로 두고두고 고마워하게 될 것이다.

최소한, 기말 고사에 같은 질문이 나왔을 때 고마워하게 될 것이다.

[5] Reader-Writer 락

또 하나의 고전적인 문제가 있다.

좀 더 융통성 있는 락 기법이 필요하다.

다양한 자료구조를 접근하는 데 여러 종류의 락 기법이 필요하다.

리스트에 삽입하고 간단한 검색을 하는 것과 같은 병행 연산이 여러 개 있다고 해보자.

삽입 연산은 리스트의 상태를 변경하고(전통적인 임계 영역 보호 방식으로 해결 가능하다.), 검색은 자료구조를 단순히 읽기만 한다.

삽입 연산이 없다는 보장만 된다면 다수의 검색 작업을 동시에 수행할 수 있다.

이와 같은 경우를 위해 만들어진 락이 reader-writer 락이다.

```
typedef struct rwlock_t {
    sem_t lock;        // 기본 세마포어
    sem_t writelock;   // 쓰기 락
    int    readers;    // reader 수
} rwlock_t;

void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // 첫 reader 가 writelock 획득
    sem_post(&rw->lock);
}
```



```

void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // 마지막 reader 가 writelock 해제
    sem_post(&rw->lock);
}

void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}

```

이 코드는 간단하다.

자료 구조를 “갱신”하려면 새로운 동기화 연산 쌍을 사용한다.

락을 획득하기 위해서 `rwlock_acquire_writelock()`을 사용하고 해제하기 위해서 `rwlock_release_writelock()`을 사용한다.

내부적으로는 `writelock` 세마포어를 사용하여 하나의 쓰기 스레드만이 락을 획득할 수 있도록 하여, 임계 영역 진입 후에 자료 구조를 갱신한다.

좀 더 재미있는 부분은 읽기 락(reader lock)의 획득과 해제이다.

읽기 락을 획득 시 읽기 스레드(reader)가 먼저 락을 획득하고 읽기 중인 스레드의 술을 표현하는 `readers` 변수를 증가시킨다.

첫 번째 읽기 스레드가 읽기 락을 획득할 때 중요한 과정이 있다.

읽기 락을 반드시 `writelock` 세마포어에 대해 `sem_wait()`을 호출하여 쓰기 락을 함께 획득한다.

획득한 쓰기 락은 읽기 락을 해제할 때 `sem_post()`로 다시 해제한다.

이 과정을 통해서 읽기락을 획득하고 난 후, 다른 읽기 스레드들이 읽기 락을 획득할 수 있도록 한다.

다만, 쓰기 락을 획득하려는 쓰기 스레드(writer)들은 모든 읽기 스레드가 끝날 때까지 대기하여야 한다.

임계영역을 빠져나오는 마지막 읽기 스레드가 `writelock`에 대한 `sem_post()`를 호출하여 대기 중인 쓰기 스레드가 락을 획득할 수 있도록 한다.

이 방식은(의도한대로) 동작하지만 몇 가지 단점이 있는데, 특히 공정성에 대한 문제가 있다. 상대적으로 쓰기 스레드가 불리한다.

쓰기 스레드에게 기아 현상이 발생하기 쉽다는 문제이다.

이에 대해서는 좀 더 세련된 해법이 존재한다.

당신도 더 좋은 구현 방법을 만들 수 있다.

힌트는 쓰기 스레드가 대기 중일 때는 읽기 스레드가 락을 획득하지 못하도록 하는 것이다.

어떻게 하면 될까? 생각해 보기 바란다.

마지막으로 읽기-쓰기 락을 사용할 때는 약간의 주의가 필요하다.

김버이 정교할수록 오버헤드가 크다는 사실이다.

때문에 단순하고 빠른 락 종류를 사용하는 것보다 항상 성능이 좋은 것은 아니다.

어쨌거나, 이제까지 세마포어를 사용하는 방법을 배웠다.

팁: 단순무식이 더 좋을 수도 있다.

단순무식한 방법이 최고가 될 수 있는 것을 절대로 과소평가해서는 안된다.

락에 있어서 때로는 간단한 스핀 락이 가장 잘 동작할 수도 있다.

왜냐하면 구현이 쉽고 간단하기 때문이다.

읽기-쓰기 락과 같은 것이 듣기에는 멋져 보일 수 있지만, 복잡하다.

복잡한 것들은 느리다. 그러므로 단순무식한 방법을 가장 먼저 시도해 보자.

간단한 것이 매력적이라는 개념은 여러 곳에서 발견할 수 있다.

이 개념에 대한 초기 주장으로 CPU의 캐시 설계를 주제로 한 Mark Hill의 학위 논문을 꼽을 수 있다. 그는 간단한 직접 매핑된 캐시가 복잡한 집합 연관 설계보다 더 잘 동작하는 것을 발견하였다. (그 이유중 하나는 간단한 설계가 캐시에서 빠른 검색을 가능하게 한다는 것이다)

Hill이 자식의 논문에서 간단하게 정리한 것처럼 “크고 무식한 것이 좋다.” 그래서 이와 비슷한 충고를 Hill's Law 라고 부른다.

[6] 식사하는 철학자

Dijkstra 가 제기한 식사하는 철학자(Dining Philosopher)라는 유명한 문제가 있다. 이 문제가 유명한 이유는 재미있기도 하고 지적인 흥미를 유발하기 때문이다.

실제 활용도는 매우 낮다.

하지만, 너무도 유명하기 때문에 이 장에서 다루어 보고자 한다.

사실, 이 문제가 면접 질문으로 나올 수도 있다.

답변을 못해서 직장을 못구했다면, 운영체제 교수를 원망할지도 모른다.

반대로, 직장을 구했다면 운영체제 교수에게 감사의 편지나 스톡옵션 보내는 것을 망설이지 마라.

이 문제는 그림에 나와있다.

다섯 명의 “철학자”가 식탁 주위를 둘러앉았다.

총 다섯개의 포크가 철학자와 철학자 사이에 하나씩 놓여 있다.

철학자는 식사하는 때가 있고 생각하는 때가 있다.

생각 중일 때는 포크가 필요없다. 자신의 왼쪽과 오른쪽에 있는 포크를 들어야 식사를 할 수 있다.

이 포크를 잡기 위한 경쟁과 그에 따른 동기화 문제가 병행 프로그래밍에서 다루려는 식사하는 철학자 문제이다.

여기 각 철학자의 동작을 나타낸 기본 반복문이 있다.

```
while (1) {  
    think();  
    getforks();  
    eat();  
    putforks();  
}
```

주요 쟁점은 getforks()와 putforks()의 루틴을 작성하되 교착 상태의 발생을 방지해야 하고, 어떤 철학자도 못 먹어서 굶주리면 안 되며 병행성이 높아야 한다. (즉, 가능한 많은 철학자가 동시에 식사를 할 수 있어야 한다.)

Downey 의 해법과 같이 문제 해결을 위한 몇 가지 함수를 사용한다.

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```

철학자 p 가 자신의 왼쪽에 있는 포크를 잡기 원한다면 $\text{left}(p)$ 를 호출한다.
마찬가지로 철학자 p 가 자신의 오른쪽에 있는 포크를 잡기 원한다면 $\text{right}(p)$ 를 호출한다.
마지막 철학자($p=4$)가 자신의 오른쪽의 포크인 포크 0 을 잡으려고 할 경우를 위해서 나머지(modulo) 연산을 사용한다.

이 문제를 해결하기 위해서 세마포어가 필요하다.
각 포크마다 한 개씩 총 다섯 개가 있고 `sem_t fork[5]`로 정의한다.

불완전한 해답

이 문제를 해결하기 위한 첫 번째 시도를 해보자.
`forks` 배열에 있는 각 포크에 대한 세마포어를 1 로 초기화 하였고, 각 철학자는 자신의 순번(p)를 알고 있다고 가정하자.

```
void getforks() {
    sem_wait(forks[left(p)]);
    sem_wait(forks[right(p)]);
}

void putforks() {
    sem_post(forks[left(p)]);
    sem_post(forks[right(p)]);
}
```

이 (불완전한) 해답의 원리는 간단하다.
포크가 필요할 때 단순히 하나의 “락”을 획득한다.
먼저 왼쪽의 것을 잡고 그 다음에 오른쪽의 것을 잡는다.
그리고 식사가 끝나면 잡은 순서대로 놓는다. 간단하지 않은가?
그래서 불완전하다는 것이다. 문제점이 보이는가? 생각해 보자.

문제는 교착 상태이다.
만약 각 철학자가 자신의 왼쪽의 포크를, 다른 철학자가 자신의 오른쪽 포크를 잡기 전에 먼저 잡았다면,
각 철학자는 하나의 포크만 들고서 다른 하나의 포크를 잡을 수 있게 되기를 평생 기다리게 된다.
구체적으로 살펴보면, 철학자 0 이 포크 0 을 잡고, 철학자 1 이 포크 1 을 잡고,
철학자 2 가 포크 2 를, 철학자 3 은 포크 3 을, 그리고 철학자 4 는 포크 4 를 잡는다.

모든 포크는 다 누군가 잡고 있기 때문에 모든 철학자는 다른 철학자가 갖고 있는 포크를 기다리며 대기하게 된다.

교착 상태에 대해서는 곧 자세히 다룰 것이다.

우선 이 방법이 제대로 동작하는 해법이 아니라는 것만 알고 넘어가자.

해답: 의존성 제거

이 문제를 해결하기 위한 가장 간단한 방법은 최소한 하나의 철학자가 다른 순서로 포크를 집도록 하면 된다.

이 방법이 Dijkstra 가 제시한 해결책이다.

가장 높은 순 번의 철학자 4 가 포크를 다른 순서로 획득한다고 가정하자.

```
void getforks() {
    if (p == 4) {
        sem_wait(forks[right(p)]);
        sem_wait(forks[left(p)]);
    } else {
        sem_wait(forks[left(p)]);
        sem_wait(forks[right(p)]);
    }
}
```

마지막 철학자가 오른쪽의 포크를 먼저 잡기 때문에 각 철학자가 하나의 포크를 든 채로 다른 포크를 기다리는 대기 상황이 발생하지 않는다.

환형 대기 상태가 끊어졌다.

이 해법에 대한 여러 상황을 한 번 생각해 보고, 동작한다는 것을 한 번 스스로 설득해 보라.

이와 비슷한 “유명한” 문제들이 더 있다.

예를 들어, 흡연가의 문제나 잠자는 이발사의 문제가 그것이다.

대부분이 병행성에 대한 연습문제 쯤 되지만, 이름 하나는 멋있다.

좀 더 알고 싶거나, 병행 프로그램 작성 연습을 하고 싶다면 참고하기 바란다.

[7] 세마포어 구현

마지막으로, 저수준 동기화 기법인 락과 컨디션 변수를 사용하여 우리만의 세마포어를 만들어 보자.

이름은 ~ 제마포어~다.

```
typedef struct Zem_t {
    int          value;
    pthread_cond_t  cond;
    pthread_mutex_t lock;
} Zem_t;

// 초기화
void Zem_init(Zem_t *s, int value) {
    s->value = value;
    Cond_init(&s->cond);
    Mutex_init(&s->lock);
}

void Zem_wait(Zem_t *s) {
    Mutex_lock(&s->lock);
    while (s->value <= 0)
        Cond_wait(&s->cond, &s->lock);
    s->value--;
    Mutex_unlock(&s->lock);
}

void Zem_post(Zem_t *s) {
    Mutex_lock(&s->lock);
    s->value++;
    Cond_signal(&s->cond);
    Mutex_unlock(&s->lock);
}
```

그림에서 보는 것과 같이 하나의 락과 하나의 컨디션 변수를 사용하고 세마포어의 값을 나타내는 상태 변수 하나를 사용한다.

스스로 코드를 이해할 수 있을 때까지 공부해 보자. 지금!

Dijkstra 가 정의한 세마포어와 여기서 정의한 제마포어 간의 중요한 차이 중 하나는 세마포어 음수 값이 대기 중인 스레드의 수를 나타낸다는 부분이다. 사실 제마포어에서는 이 값이 0 보다 작을 수가 없다. 이 방식이 구현하기도 쉽고 현재 Linux 에 구현된 방식이기도 하다.

세마포어를 사용하여 락과 컨디션 변수를 만드는 것은 까다로운 문제이다. 병렬 연산에 많은 경험이 있는 개발자가 Windows 운영체제 환경에서 이것을 시도했지만 버그가 무척 많이 발생했다. 한 번 스스로 시도해 보면, 세마 포어를 사용하여 컨디션 변수를 구현하는 것이 생각보다 어렵다는 것을 알게 될 것이다. 한 번 더 강조하지만, 정말 쉽지 않다. 세마포어로 컨디션 변수 구현에 성공한다면, 당신은 뛰어난 개발자의 소양을 갖고 있는 것이다.

팁: 일반화를 주의하자

좋은 개념을 약간 확장하여 여러 부류의 문제를 해결할 수 있도록 하는 “일반화”라는 기법은 시스템 설계에 아주 유용하다.

하지만, 일반화를 할 때 주의해야 한다.

Lampson 이 경계한 것처럼 “일반화를 하지말자. 일반적으로 일반화는 틀렸다.”

어떤 이는 세마포어를 락과 컨디션 변수의 일반화로 볼 수도 있다.

하지만, 그런 일반화가 필요하긴 한가?

그리고 세마포어를 이용하여 컨디션 변수를 구현하는 것이 어렵다는 것을 바탕으로 생각하면, 이 일반화가 실제 당신이 생각하는 것만큼 일반적이지 않을 수도 있다.

[1-2] 병행성: 병행성 관련 오류

수년간 연구자들은 병행성 버그를 해결하기 위해 엄청난 시간과 노력을 쏟아왔다. 초기에는 주로 교착 상태(deadlock)에 집중했지만, 최근 연구들은 그 외의 다양한 병행성 버그도 다루고 있다.

이 장의 핵심 질문은 다음과 같다.

- 일반적인 병행성 오류들은 어떻게 처리할 것인가?

[1] 오류의 종류

Lu 등의 연구팀은 실제 많이 쓰이는 오픈소스 프로그램 4 개를 분석했다.

응용 프로그램	역할	비 교착 상태	교착 상태
MySQL 데이터베이스 서버	데이터베이스 서버	14	9
Apache 웹 서버	웹 서버	13	4
Mozilla 웹 브라우저	웹 브라우저	41	16
OpenOffice 오피스 모음	오피스 모음	6	2
합계	-	74	31

총 104 개의 버그 중 74 개(약 70%)가 비 교착 상태 오류였다.

즉, 교착 상태보다 그 외의 오류가 훨씬 더 많이 발생한다는 것이 핵심 발견이다.

[2] 비 교착 상태 오류

[2-1] 원자성 위반(Atomicity Violation)

개념: “여러 동작이 끊임 없이 한 번에 실행되어야 한다”는 가정이 깨질 때 발생한다.

문제 상황(MySQL 실제 버그):

쓰레드 1: proc_info 가 NULL 인지 확인 → (여기서 끊임!) → fputs()로 출력

쓰레드 2: proc_info = NULL 로 설정

쓰레드 1 이 NULL 이 아님을 확인한 후, 쓰레드 2 가 끼어들어 NULL 로 바꿔버리면 쓰레드 1 은 NULL 포인터를 역참조해 프로그램이 크래시된다.

원인: “확인”과 “사용”이 원자적으로 실행된다고 가정했지만, 실제로는 그 사이에 다른 쓰레드가 끼어들 수 있다.

해결 - 락(Lock)으로 두 동작을 묶기

```
pthread_mutex_lock(&proc_info_lock);
if (thd->proc_info) {
    fputs(thd->proc_info, ...);
}
pthread_mutex_unlock(&proc_info_lock);
```

두 동작을 락으로 감싸면 그 사이에 다른 스레드가 끼어들 수 없다.

[2-1] 순위 위반(Order Violation)

개념: “A 는 반드시 B 보다 먼저 실행되어야 한다”는 순서가 보장되지 않을 때 발생한다.

문제 상황:

```
스레드 1 (init): mThread = PR_CreateThread(...) ← 나중에 실행될 수도 있음
스레드 2 (mMain): mState = mThread->State          ← mThread 가 NULL 이면 크래시!
```

스레드 2 는 스레드 1 이 이미 mThread 를 초기화했다고 가정하지만, 실행 순서는 보장되지 않는다.

해결 - 컨디션 변수(Condition Variable)로 순서 강제:

```
// 스레드 1: 초기화 완료 후 신호 발송
mThread = PR_CreateThread(...);
pthread_mutex_lock(&mtLock);
mtInit = 1;
pthread_cond_signal(&mtCond); // "나 끝났어!" 신호
pthread_mutex_unlock(&mtLock);

// 스레드 2: 신호 올 때까지 대기
pthread_mutex_lock(&mtLock);
while (mtInit == 0)
    pthread_cond_wait(&mtCond, &mtLock); // 신호 기다림
pthread_mutex_unlock(&mtLock);
mState = mThread->State; // 이제 안전
```

정리: 비교착 상태 오류의 97%가 이 두 종류(원자성 위반 + 순서 위반)에 해당한다.

패턴을 알면 코드 작성 시 미리 조심할 수 있고, 자동화 검사 도 구로도 탐지 가능성이 높다.

[2-3] 교착 상태 오류(Deadlock)

교착 상태란?

쓰레드 1: L1 획득 → L2 를 기다림
쓰레드 2: L2 획득 → L1 을 기다림
→ 서로 상대방 것을 기다리며 영원히 멈춤

의존성 그래프에 사이클(순환 고리)이 생기면 교착 상태가 발생한다.

발생 이유: 두 가지 주요 원인

- (1) 복잡한 의존성: 코드가 커질수록 구성 요소 간의 의존 관계가 복잡해진다.
예를 들어, 가상 메모리 시스템이 파일 시스템을 부르고, 파일 시스템이 다시 가상 메모리 시스템을 부르는 순환 구조가 자연스럽게 생긴다.
- (2) 캡슐화: 내부 구현을 감추는 모듈화는 좋은 원칙이지만, 락과는 잘 맞지 않다.
예를 들어, Java 의 `v1.AddAll(v2)`와 `v2.AddAll(v1)`이 거의 동시에 호출되면, 내부적으로 락 획득 순서가 반대가 되어 교착 상태가 발생할 수 있다. 이 모든 것이 호출한 쪽에서는 전혀 보이지 않는다.

교착 상태 발생 4 가지 조건(Coffman 조건)

교착 상태는 아래 4 가지 조건이 모두 동시에 충족될 때 발생한다.

하나라도 개면 교착 상태는 일어나지 않는다.

조건	의미
상호 배제	자원을 한 쓰레드가 독점적으로 사용
점유 및 대기	자원을 이미 가지고 있으면서 다른 자원을 추가로 기다림
비선점	다른 쓰레드의 자원을 강제로 빼앗을 수 없음
환형 대기	쓰레드들이 서로의 자원을 기다리는 순환 구조 형성

교착 상태 대응 전략

전략 1. 예방 - 순환 대기 제거(가장 실용적)

방법: 모든 락에 획득 순서를 미리 정해두기

// 항상 L1 → L2 순서로만 획득
// 어떤 쓰레드든 이 순서를 지키면 순환 고리가 생기지 않음

락이 여러 개 일 때는 락의 메모리 주소 크기로 순서 결정하는 트릭도 유효하다.

```
if (m1 > m2) { lock(m1); lock(m2); }  
else { lock(m2); lock(m1); }  
// 인자 순서와 무관하게 항상 같은 순서로 획득됨
```

단, 순서는 관례일 뿐이라 개발자가 모르고 어기면 교착 상태가 발생할 수 있다.

전략 2. 예방 - 점유 및 대기 제거

방법: 전역 prevention 락으로 모든 락을 한꺼번에 획득

```
lock(prevention); // 전역 락 먼저 획득
lock(L1);
lock(L2);
unlock(prevention);
```

단점: 필요한 락을 미리 다 파악해야 하고, 병행성이 떨어진다.

전략 3. 예방 - 비선점 제거(trylock 활용)

방법: 락 획득 실패 시 갖고 있던 락도 반납하고 재시도

```
top:
lock(L1);
if (trylock(L2) == -1) { // L2 획득 실패 시
    unlock(L1);          // L1 도 반납
    goto top;            // 처음부터 재시도
}
```

단점: 무한 반복(livelock) 가능성이 있다. 두 쓰레드가 계속 같은 타이밍에 실패를 반복할 수 있다. 이를 줄이기 위해 재시도 전 무작위 대기 시간을 넣는 방법을 쓴다.

전략 4. 예방 - 상호배제 제거(Compare-And-Swap)

방법: 락 자체를 없애고 하드웨어 원자 명령어로 대체

```
// 값을 원자적으로 증가 — 락 없이도 안전
void AtomicIncrement(int *value, int amount) {
    do {
        int old = *value;
    } while (CompareAndSwap(value, old, old + amount) == 0);
}
```

락이 없으므로 교착 상태 자체가 불가능하다. 단, 복잡한 자료 구조에 적용하기 어렵고 무한 반복 가능성은 여전히 있다.

전략 5. 회피 - 스케줄링으로 충돌 방지

방법: 어떤 쓰레드가 어떤 락을 쓰는지 미리 파악해서, 충돌 가능한 쓰레드는 동시에 실행되지 않도록 스케줄링한다.(Dijkstra 의 은행원 알고리즘 등)

단점: 모든 락 사용 정보를 미리 알아야하고, 병행성이 크게 제한된다. 임베디드 시스템처럼 환경이 고정된 경우에만 유효하다.

전략 6. 발견 및 복구

방법: 교착 상태를 허용하되, 주기적으로 자원 할당 그래프의 사이클을 탐지해 발견 시 재시작 한다. 데이터베이스 시스템에서 많이 사용하는 현실적인 방법이다.

팁: Tom West 의 법칙

“해야 하는 모든 일을 완벽하게 할 필요는 없다.” 교착 상태가 아주 드물게 발생하고 피해가 작다면, 완벽한 예방에 큰 비용을 쓰기보다 재부팅으로 복구하는 편이 더 실용적일 수 있다. 물론 우주선처럼 실패가 치명적인 경우라면 얘기가 다르다.

[4] 요약

분류	종류	해결 방법
비 교착 상태	원자성 위반	락으로 관련 코드 묶기
비 교착 상태	순서 위반	컨디션 변수로 실행 순서 강제
교착 상태	순환 대기	락 획득 순서 고정
교착 상태	점유 및 대기	락을 한 번에 모두 획득
교착 상태	비선점	trylock + 실패 시 재시도
교착 상태	상호 배제	CAS 등 락-프리 자료구조 사용
교착 상태	회피 / 복구	스케줄링 또는 감지 후 재시작

가장 현실적이고 널리 쓰이는 방법은 락 획득 순서를 미리 정해두는 것이며, 장기적으로는 MapReduce 처럼 락 자체를 쓰지 않는 새로운 프로그래밍 모델이 근본적인 해결책으로 주목받고 있다.

[1-3] 병행성: 이벤트 기반 병행성(Event-Based Concurrency)

왜 이벤트 기반인가?

기존 멀티스레드 방식에는 두 가지 근본적인 문제가 있다.

1. 버그 위험 - 락 누락, 교착 상태 등 디버깅이 매우 어려운 문제들이 발생하기 쉽다.
2. 스케줄링 제어 불가 - 개발자가 스레드 실행 순서를 통제할 수 없고, OS 스케줄러에 완전히 의존해야 한다.

그래서 나온 대안이 이벤트 기반 병행성이다.

node.js 가 대표적인 예시이고, 원래 C/Unix 시스템에서 시작된 개념이다.

[1] 핵심 구조: 이벤트 루프

```
while (1) {  
    events = getEvents();      // 이벤트 발생을 기다림  
    for (e in events)  
        processEvent(e);      // 하나씩 처리  
}
```

구조 자체는 단순하다.

- 이벤트 “기다리기 -> 처리하기”의 반복이다.

여기서 중요한 포인트는 다음에 어떤 이벤트를 처리할지 개발자가 직접 결정할 수 있다.

이게 곧 스케줄링을 제어하는 것과 동일한 효과를 보인다.

멀티스레드에서는 OS가 알아서 하던걸, 여기서는 내가 코드로 결정하는 것이다.

[2] 핵심 API: select() / poll()

select()가 하는일

select()의 역할은 단순하다.

현재 처리 가능한 I/O가 있는지 검사하는 것이다. 예를 들어, 웹 서버가 “내가 처리해야 할 패킷이 도착했나?”를 확인할 때 쓰는 함수이다.

함수 구조

```
int select(int nfds,  
           fd_set *readfds, // 읽을 준비된 fd 집합  
           fd_set *writefds, // 쓸 준비된 fd 집합  
           fd_set *errorfds, // 에러 발생한 fd 집합  
           struct timeval *timeout);
```

select()는 세 가지 fd(파일 디스크립터) 집합을 받아서, 0번부터 nfds-1번까지의 디스크립터 중 지금 당장 처리 가능한 것들만 골라서 돌려준다.

반환 값은 준비된 디스크립터의 개수이다.

알아두어야 할 두가지

(1) 읽기 / 쓰기 여부를 모두 감시할 수 있다.

- readfds: 수신 패킷이 도착했는지 확인하는 용도
- writefds: 아웃바운드 큐에 여유가 생겨서 응답을 보낼 수 있는지 확인하는 용도

(2) timeout 인자로 대기 방식을 조절할 수 있다.

- NULL: 준비된 fd가 생길 때까지 무한 대기한다.
- 특정시간: 해당 시간만큼만 기다린다.
- 0: 즉시 반환한다(polling 방식)

인자	의미
readfds	읽을 데이터가 도착했는지 확인 (예: 패킷 수신)
writefds	지금 데이터를 보낼 수 있는지 확인 (예: 출력 큐에 여유 있음)
timeout	NULL이면 무한 대기, 0이면 즉시 반환

차단(Blocking) vs 비차단(Non-blocking)

구분	Blocking	Non-blocking
동작	작업 완료까지 기다림	즉시 반환, 작업은 백그라운드에서 진행
예시	read() 호출 후 데이터 올 때까지 멈춤	요청만 보내고 다른 작업 계속 수행
이벤트 기반 시스템	❌ 사용하면 안 됨	✅ 필수

Blocking(차단) 방식

작업이 끝날 때까지 호출자가 멈춰서 기다리는 방식이다. 디스크 I/O가 대표적인 예시이다.

Non-blocking(비차단) 방식

요청만 하고 즉시 반환하며, 실제 작업은 백그라운드에서 완료되는 방식이다.

이벤트 기반 방식에서 blocking 호출은 절대 사용해서는 안된다.

단일 루프 구조이기 때문에, 하나의 호출이 블락되면 전체 시스템이 멈춰버리기 때문이다. 그래서 non-blocing 이 필수이다.

[3] select()의 사용

코드 구조 설명

select()를 활용한 기본적인 이벤트 서버의 흐름은 다음과 같다.

```
while (1) {  
    // 1. fd_set 초기화  
    fd_set readFDs;  
    FD_ZERO(&readFDs);  
  
    // 2. 감시할 fd 들을 집합에 등록  
    for (fd = minFD; fd < maxFD; fd++)  
        FD_SET(fd, &readFDs);  
  
    // 3. select()로 준비된 fd 가 있는지 검사  
    int rc = select(maxFD+1, &readFDs, NULL, NULL, NULL);  
  
    // 4. 준비된 fd 만 골라서 처리  
    for (fd = minFD; fd < maxFD; fd++)  
        if (FD_ISSET(fd, &readFDs))  
            processFD(fd);  
}
```

동작 순서는 총 네 단계이다.

- (1) FD_ZERO()로 fd 집합을 초기화한다.
- (2) FD_SET()으로 감시할 fd 들을 집합에 등록한다. 서로가 열어둔 모든 네트워크 소켓이 이 집합에 들어간다.
- (3) select()를 호출하여 그 중 데이터가 도착한 fd 가 있는지 OS 에게 물어본다.
- (4) FD_ISSET()으로 실제로 준비된 fd 만 골라내어 처리한다.

핵심 팁: 이벤트 기반 서버 안에서는 절대 블락하지 말아야 한다.

이벤트 기반 서버의 강점은 작업 스케줄링을 정밀하게 제어할 수 있다는 점이다.

그런데 어딘가에서 blocking 호출이 발생하면 루프 전체가 멈춰버린다.

따라서 이벤트 핸들러 내부에서는 blocking 을 유발하는 호출을 절대 사용해서는 안된다.

[4] 왜 간단한가? 락이 필요 없다

단일 CPU 기반의 이벤트 서버에서는 멀티스레드 프로그램에서 겪었던 복잡한 문제들이 사라진다.

그 이유는 명확하다.

매 순간 단 하나의 이벤트만 처리하기 때문에, 여러 스레드가 공유 자원에 동시에 접근하는 상황 자체가 발생하지 않는다.

따라서 락을 획득하거나 해제할 필요가 없다.

또한 이벤트 기반 서버는 단 하나의 스레드만 가지기 때문에, 다른 스레드에 의해 실행 중단(인터럽트)되는 일도 없다.

결과적으로 멀티스레드 환경에서 흔히 발생하는 경쟁 조건(race condition)이나 교착 상태(deadlock) 같은 병행성 버그가 이벤트 기반 방식에서는 근본적으로 나타나지 않는다.

[5] 문제: 블로킹 시스템 콜

문제 상황

이벤트 기반 방식은 락도 필요 없고 구조도 단순해 보이지만, 치명적인 약점이 하나 있다. 바로 blocking 시스템 콜을 호출해야 하는 상황이다.

예를 들어, 클라이언트가 디스크의 파일을 요청하면, 이벤트 핸들러가 open()으로 파일을 열고 read()로 내용을 읽어야 한다.

그런데 해당 데이터가 메모리에 없어서 실제 디스크 I/O가 발생하면, 그 작업이 끝날 때까지 오랜 시간이 걸린다.

방식	블로킹 I/O 발생 시
멀티스레드	해당 스레드만 대기, 나머지 스레드는 계속 실행
이벤트 기반	루프 전체가 멈춤 → 모든 이벤트 처리 중단

이벤트 기반 서버는 스레드가 하나 뿐이기 때문에, blocking 호출 하나가 시스템 전체를 유휴 상태로 만들어버리는 심각한 자원 낭비가 발생한다.

[6] 해법: 비동기 I/O

비동기 I/O 란

이 문제를 해결하기 위해 현대 운영체제들은 비동기 I/O(Asynchronous I/O)를 제공한다. 핵심은 I/O 요청을 보낸 뒤 완료를 기다리지 않고 즉시 제어권을 돌려받는 것이다.

Mac OS X 의 비동기 I/O API

비동기 I/O 를 사용하려면 먼저 aiocb 구조체에 요청 정보를 채운다.

```
struct aiocb {
    int    aio_fildes;    // 파일 디스크립터
    off_t  aio_offset;    // 파일 내 읽기 시작 위치
    void   *aio_buf;      // 읽은 데이터를 저장할 버퍼
    size_t aio_nbytes;    // 읽을 데이터 크기
};
```

구조체를 채운 후, aio_read()를 호출하면 I/O 요청이 시작되고, 완료를 기다리지 않고 즉시 반환된다.

서버는 그 사이에 다른 이벤트를 계속 처리할 수 있다.

```
int aio_read(struct aiocb *aiocbp);    // 즉시 반환
```

I/O 완료 여부 확인

I/O 가 끝났는지 확인하는 방법은 두 가지이다.

(1) 폴링(Polling) 방식 - aio_error()로 주기적으로 완료 여부를 직접 물어본다.

```
int aio_error(const struct aiocb *aiocbp);
// 완료 → 0 반환
// 진행 중 → EINPROGRESS 반환
```

(2) 인터럽트(Interrupt) 방식 - 유닉스 시그널(signal)을 사용하여, I/O 가 완료되면 OS 가 응용 프로그램에 직접 알려준다. 반복적으로 확인할 필요가 없다.

비동기 I/O 가 없는 경우의 대안

비동기 I/O 를 지원하지 않는 시스템에서는 하이브리드 방식이 사용된다.

네트워크 이벤트 처리는 이벤트 기반으로 하되, 디스크 I/O 처럼 블로킹이 불가피한 작업은 별도의 스레드 풀(thread pool)에 맡기는 방식이다.

이벤트 기반의 장점을 살리면서 blocking 문제도 우회할 수 있는 현실적인 절충안이다.

[7] 또 다른 문제점: 상태 관리

문제의 본질

이벤트 기반 방식은 스레드 기반 보다 코드 작성이 더 복잡하다.

그 이유는 비동기 I/O 를 요청한 뒤, I/O 가 완료됐을 때 “다음에 뭘 해야 하는지”를 개발자가 직접 저장해둬야 하기 때문이다.

스레드 기반에서는 이 문제가 없다.

스레드 스택에 모든 상태(지역 변수, 함수 호출 흐름 등)가 자동으로 보존되기 때문이다.

반면 이벤트 기반에서는 스택이 하나 뿐이고 이벤트 핸들러가 반환되면 그 정보가 사라진다.

이것을 수동 스택 관리(manual stack management)라고 부른다.

비교: 스레드 기반 vs 이벤트 기반

스레드 기반에서는 파일을 읽어 소켓으로 전송하는 코드가 다음처럼 단순하다.

```
int rc = read(fd, buffer, size); // 블로킹, 완료까지 대기
rc = write(sd, buffer, size);    // 바로 이어서 전송
```

read()가 끝나면 같은 스택 위에 sd(소켓 디스크립터) 정보가 그대로 남아 있으므로 바로 write()를 호출하면 된다.

이벤트 기반에서는 read()를 비동기로 요청하고 즉시 반환된다.

나중에 I/O 완료 이벤트가 왔을 때, “이 결과를 어느 소켓으로 보내야 하는지”를 알 방법이 없다.

이미 핸들러가 종료됐기 때문이다.

해법: Continuation(연속성)

해법은 Continuation 이라는 개념이다. 복잡하게 들리지만 개념은 단순하다.

“나중에 이벤트가 완료됐을 때 필요한 정보를 미리 어딘가에 저장해두는 것”이다.

위의 예시에서는 fd(파일 디스크립터)를 키로 하는 해시 테이블에 sd(소켓 디스크립터)를 저장해두는 방식으로 해결한다.

비동기 read() 요청 시점:

해시 테이블[fd] = sd ← 나중에 쓸 정보를 저장

I/O 완료 이벤트 발생 시점:

sd = 해시 테이블[fd] ← 저장해둔 정보를 꺼내서

write(sd, buffer, size) ← 전송 완료

구분	스레드 기반	이벤트 기반
상태 보존 방식	스택이 자동으로 유지	개발자가 직접 상태 저장
복잡도	낮음	높음
해법	별도 기법 필요 없음	Continuation 패턴 사용

이벤트 기반 방식은 락 문제를 피할 수 있지만, 그 대가로 상태를 수동으로 관리해야 하는 복잡성을 개발자가 부담하게 된다.

[8] 이벤트 사용의 어려움

어려움 1. 멀티 코어 환경에서 단순함이 사라지다.

단일 CPU 에서는 한 번에 하나의 이벤트만 처리하기 때문에 락이 필요 없었다.

그런데 멀티코어 환경에서 성능을 제대로 활용하려면 여러 이벤트 핸들러를 병렬로 실행해야 한다.

그 순간 오유 자원에 대한 동기화 문제가 발생하고, 결국 락을 다시 사용해야 한다.

이벤트 기반의 핵심 장점이었던 “락 불필요”가 사라지는 것이다.

어려움 2. 페이지 폴트가 발생하면 전체가 멈춘다.

이벤트 핸들러 실행 중 페이지 폴트가 발생하면, 해당 페이지가 메모리에 올라올 때까지 서버 전체가 중단된다.

non-blocking 으로 아무리 잘 설계해도 이런 OS 내부에서 발생하는 암묵적 blocking 은 피하기가 어렵다.

이런 상황이 자주 발생하면 심각한 성능 저하로 이어진다.

어려움 3.API 가 바뀌면 핸들러도 전부 바꿔야 한다.

어떤 루틴이 non-blocking 에서 blocking 방식으로 변경되면, 그 루틴을 호출하는 모든 이벤트 핸들러도 함께 수정해야 한다.

이벤트 기반 서버에서 blocking 은 치명적이기 때문에, 개발자는 사용하는 모든 API 의 동작 방식 변화를 항상 예의 주시해야 한다.

이는 유지보수를 매우 어렵게 만든다.

어려움 4. 비동기 I/O 의 일관성 부족

비동기 디스크 I/O 는 지원되기까지 오랜 시간이 걸렸고, 비동기 네트워크 I/O 는 아직도 플랫폼마다 일관성이 부족하다.

이상적으로는 모든 I/O 를 select() 하나로 통일하는 것이 좋지만, 현실에서는 네트워크는 select(), 디스크는 AIO 를 따로 사용해야 하는 불일치가 존재한다.

[9] 요약

이벤트 기반 병행성의 장단점을 정리하면 다음과 같다.

구분	스레드 기반	이벤트 기반
상태 보존 방식	호출 스택(Call Stack)이 자동으로 유지됨	개발자가 상태를 직접 저장 및 관리해야 함
복잡도	낮음 (직관적인 순차 실행)	높음 (비동기 흐름 관리 필요)
해법	별도의 구조 필요 없음	Continuation 패턴, 콜백, async/await 사용

결국 이벤트 기반 방식도 완전한 해답은 아니다.

그렇기 때문에 스레드 기반과 이벤트 기반은 병행성 문제에 대한 두 가지 서로 다른 접근법으로 오랫동안 공존하게 될 것이다.